

TP : Communication, Sécurité et Contrôle d'un Robot Mobile sous ROS 2

Filière Robotique

Mai 2026

Table des matières

1	Exercice 1 : Déplacement rectiligne simple par Odométrie	3
1.1	Idée générale	3
1.2	Travail demandé	3
2	Exercice 2 : Enchaînement de mouvements (Translation et Rotation)	4
2.1	Idée générale	4
2.2	Travail demandé	4
3	Exercice 3 : Intégration du LiDAR et Arrêt d'Urgence	5
3.1	Idée générale	5
3.2	Travail demandé	5
4	Exercice 4 : Exploration Autonome par Machine à États	6
4.1	Idée générale	6
4.2	Travail demandé	6
5	Exercice 5 : Suivi de Couloir Réactif et Régulation PD	7
5.1	Idée générale	7
5.2	Travail demandé	7

Introduction et Objectifs du TP

L'objectif de cette série de travaux pratiques est de comprendre comment interagir avec un robot mobile autonome en utilisant le middleware **ROS 2** et le langage **Python**.

À travers 5 exercices de complexité croissante, vous apprendrez à récupérer les données des capteurs du robot (Odométrie, LiDAR) et à lui envoyer des commandes de mouvement. Pour chaque exercice, vous devrez récupérer un code "à trous" sur le dépôt GitHub de l'enseignant, analyser sa structure, répondre aux questions théoriques, puis compléter les calculs et variables manquants pour faire fonctionner le robot.

Accès aux fichiers du TP

Téléchargez les fichiers de départ pour chaque exercice à l'adresse suivante :
<https://github.com/MatthiasVermot-Desroches/Polytech-Scout-Agilex-GR-TP/tree/main/Chavaroche/Document%20Code/suiveur2lignes/test>

Rappels : Boîte à outils ROS 2 (rclpy)

Pour vous aider dans la compréhension du code, voici un récapitulatif des fonctions spécifiques à ROS 2 et à la gestion des données de capteurs utilisées dans ce TP :

- `rclpy.init()` Initialise le système de communication ROS 2 pour ce script. C'est obligatoirement la première ligne de code ROS 2 exécutée.
- `super().__init__('nom_du_noeud')` Initialise la classe parente (Node). C'est elle qui donne officiellement un nom au nœud dans le réseau ROS 2 (visible via la commande `ros2 node list`).
- `self.create_publisher(Type, 'topic', queue_size)` Crée une ligne de communication sortante. Le robot l'utilise pour publier (envoyer) ses commandes de vitesse sur le topic `/cmd_vel`.
- `self.create_subscription(Type, 'topic', callback, queue_size)` Crée une ligne de communication entrante. Chaque fois qu'un message arrive sur le topic (ex : odométrie ou LiDAR), ROS 2 met le script en pause l'espace d'un instant pour exécuter la fonction de `callback` associée.
- `self.create_timer(0.1, self.ma_fonction)` Configure une alarme logicielle. ROS 2 va appeler la fonction spécifiée de manière ultra-régulière à intervalle de temps fixe (ici 0.1 seconde, soit une fréquence de 10 Hz).
- `self.get_logger().info("...")` L'équivalent du `print()` en Python, mais propre à ROS. Il affiche le message dans le terminal avec un horodatage précis et le nom du nœud, et permet de l'enregistrer dans des fichiers de log.
- `pc2.read_points(msg, field_names=..., skip_nans=True)` Fonction issue du module `sensor_msgs_py`. Un LiDAR 3D renvoie des milliers de points sous forme de données binaires brutes (des octets). Cette fonction sert de "décodeur" pour extraire proprement les coordonnées (X, Y, Z) de chaque point. L'argument `skip_nans=True` permet d'ignorer les points où le laser a tiré dans le vide.
- `rclpy.spin(robot)` C'est le "moteur" asynchrone de ROS 2. Cette fonction crée une boucle infinie invisible qui écoute le réseau. C'est elle qui déclenche les **callbacks** des souscriptions quand un message arrive, ou les fonctions des **timers** quand le temps est écoulé. Elle garde le script en vie jusqu'à ce qu'on fasse **Ctrl+C**.
- `robot.destroy_node()` et `rclpy.shutdown()` Nettoient proprement la mémoire, coupent les connexions réseau du nœud et ferment proprement ROS 2 avant de quitter le script Python.

1 Exercice 1 : Déplacement rectiligne simple par Odométrie

1.1 Idée générale

Le but de ce premier programme est d'apprendre à faire avancer le robot en ligne droite sur une distance précise demandée à l'utilisateur. Le programme doit écouter la position actuelle du robot et interrompre le mouvement dès que la cible est atteinte.

1.2 Travail demandé

1. Récupérez le script `robot_simple_ex1.py` sur le dépôt GitHub.
2. À votre avis, à quoi servent les deux *topics* configurés dans le constructeur (`__init__`) ? Quel est le rôle respectif de `/cmd_vel` et `/odom` ?
3. Dans la fonction `distance_parcourue()`. Quelle formule mathématique standard peut-on utiliser ici pour suivre le déplacement du robot ?
4. A votre avis, comment la boucle est-elle appelé dans le code.
5. **À compléter dans le code :** Remplissez la condition mathématique dans la fonction `boucle()` qui permet de valider si le robot doit continuer à avancer ou s'arrêter.
6. Finalement, expliquez les étapes successives fonctionnement du code. Soyez précis et utilisez les fonctions et variables du code pour votre explication.

Finalement vous devriez obtenir :



```
Combien de mètres ? 0.2
[INFO] [1782892777.892899857] [robot_simple]: Arrivé !
```

FIGURE 1 – Resultat Test_1

2 Exercice 2 : Enchaînement de mouvements (Translation et Rotation)

2.1 Idée générale

Maintenant que le robot sait avancer, nous voulons lui permettre de tourner d'un angle (en degrés) défini par l'utilisateur. L'objectif est de pouvoir combiner une distance et un angle pour commencer à dessiner des trajectoires simples.

2.2 Travail demandé

1. Récupérez le script `robot_simple_ex2.py` sur le dépôt GitHub.
2. Observez les variables d'état ajoutées dans le constructeur (`self.avance`, `self.en_rotation`, etc.). Pourquoi a-t-on besoin de séparer les états de mouvement et de rotation ?
3. Comment la fonction `angle_parcouru()` estime-t-elle le pivotement du robot à partir des données de l'odométrie ? Quelle est la limite de cette méthode simplifiée ?
4. Expliquez le rôle de la boucle `while robot.en_mouvement or robot.en_rotation:` située dans le `main()`. Que se passerait-il si on la supprimait ?
5. **À compléter dans le code :** Complétez la boucle afin que le robot effectue les translations et rotations.
6. Finalement, expliquez les étapes successives fonctionnement du code. Soyez précis et utilisez les fonctions et variables du code pour votre explication.

Finalement vous devriez obtenir :

```
Combien de mètres ? 0.2
Combien de degrés ? 0
[INFO] [1782892910.261959549] [robot_simple]: Arrivé !
[INFO] [1782892910.328253053] [robot_simple]: Rotation terminée !
Arrivé ! Prochaine commande ?
Continuer ? (o/n) : n
```

FIGURE 2 – Resultat Test_2

3 Exercice 3 : Intégration du LiDAR et Arrêt d'Urgence

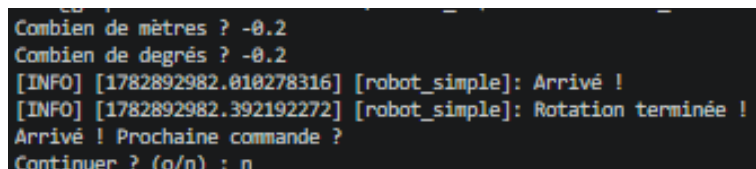
3.1 Idée générale

Faire bouger un robot à l'aveugle est dangereux. Nous intégrons ici un capteur **LiDAR 3D** qui renvoie un nuage de points de l'environnement. Le but est d'analyser ce nuage pour détecter si un obstacle se trouve sur la trajectoire du robot (devant ou derrière) et de couper instantanément les moteurs en cas de danger.

3.2 Travail demandé

1. Récupérez le script `robot_simple_ex3.py` sur le dépôt GitHub.
2. Quel est le type de message ROS 2 utilisé pour recevoir les données du LiDAR ? Pourquoi utilise-t-on la fonction `pc2.read_points()` ?
3. Dans `lidar_callback()`, à quoi sert le filtre appliqué sur la coordonnée `z` (`z < -0.1 or z > 1.5`) ? Qu'arriverait-il si on ne filtrait pas selon cette composante ?
4. Expliquez la condition géométrique `abs(y) < self.largeur_robot`. Pourquoi est-elle indispensable pour éviter que le robot ne s'arrête à cause d'un obstacle situé sur le côté (par exemple un mur parallèle) ?
5. **À compléter dans le code :** Complétez la boucle afin d'utiliser les données lidars quand le robot est en mouvement.
6. Finalement, expliquez les étapes successives fonctionnement du code. Soyez précis et utilisez les fonctions et variables du code pour votre explication.

Finalement vous devriez obtenir :



```
Combien de mètres ? -0.2
Combien de degrés ? -0.2
[INFO] [1782892982.010278316] [robot_simple]: Arrivé !
[INFO] [1782892982.392192272] [robot_simple]: Rotation terminée !
Arrivé ! Prochaine commande ?
Continuer ? (o/n) : n
```

FIGURE 3 – Resultat Test_3

4 Exercice 4 : Exploration Autonome par Machine à États

4.1 Idée générale

Le robot doit maintenant devenir totalement autonome dans son environnement. Pour y parvenir sans bloquer l'exécution globale du nœud ROS 2, nous abandonnons l'approche séquentielle. Les fonctions bloquantes `avancer()` et `tourner()` sont supprimées au profit d'une **Machine à États Finis (FSM)** centralisée dans la méthode périodique `boucle()`.

Le robot adopte par défaut l'état "AVANCER". Dès qu'un obstacle est détecté par le LiDAR à moins de 80 cm, le robot calcule la direction latérale la plus dégagée, initialise ses variables de cible odométrique, et bascule instantanément dans l'état "TOURNER". Une fois le pivot sur place terminé, il reprend sa marche avant. La gestion temporelle du script est confiée à un exécuteur asynchrone (`rclpy.spin()`).

4.2 Travail demandé

1. Récupérez le script `robot_simple_ex4.py` sur le dépôt GitHub de l'UE.
2. **Analyse du filtrage LiDAR** : Observez attentivement la fonction `lidar_callback()`.
 - Comment les 4 secteurs de détection (*devant, derrière, gauche, droite*) sont-ils isolés géométriquement à l'aide des coordonnées X et Y des points laser ?
 - Pourquoi le filtre sur l'axe Z (`if z < -0.5 or z > 0.2`) est-il indispensable pour un LiDAR 3D multicouches (type RS-Helios) monté sur le châssis du robot ?

VERIFIER LE MIEUX ENTRE LES VALEURS DE L'EXO 3 ET CELUI LA

3. Comment la méthode `meilleure_direction()` parvient-elle à déterminer de quel côté tourner ? Quelle fonction intégrée (*built-in*) de Python et quel argument spécifique sont exploités ici pour trier le dictionnaire ?
4. **Intérêt de la Machine à États (FSM)** : Pourquoi l'utilisation d'une structure en "Machine à États" (`self.etat = "..."`) est-elle particulièrement pertinente ici pour l'autonomie du robot, plutôt qu'une simple suite de conditions `if/else` classiques ou de fonctions séquentielles ? Quels avantages cela apporte-t-il en termes de réactivité face aux obstacles et d'évolution du code ?
5. **À compléter dans le code** : Ouvrez le script `robot_simple_ex4.py`. La fonction principale `boucle(self)` a été vidée de sa logique algorithmique. En vous aidant des commentaires présents dans le code et des variables d'état du robot, complétez les zones manquantes pour que le robot gère correctement ses transitions d'états et l'envoi de ses commandes de vitesse (`Twist`).

Finalement vous devriez obtenir :

```
user@gr-p247-humble-container:~/roskit_ws$ ros2 run robot_control test_4
Le robot explore la pièce en mode Machine à États - Ctrl+C pour arrêter
```

FIGURE 4 – Resultat Test_4

5 Exercice 5 : Suivi de Couloir Réactif et Régulation PD

5.1 Idée générale

Pour rendre les déplacements du robot fluides et lui permettre de centrer sa trajectoire au milieu d'un couloir ou de longer un mur sans osciller, nous abandonnons les consignes de vitesse fixes. Ce script introduit un correcteur **Proportionnel-Dérivé (PD)** qui adapte la vitesse angulaire (`cmd_z`) en continu selon la géométrie mesurée par le LiDAR.

De plus, le robot calcule désormais une **erreur d'anticipation** pour « lire la route » en diagonale et ralentit dynamiquement (`ratio_vitesse`) à l'approche d'un obstacle frontal. Enfin, pour préserver les moteurs de la plateforme physique et éliminer les secousses, les commandes de vitesse subissent un **lissage exponentiel** (filtre passe-bas logiciel). Une machine à états hiérarchisée gère les contextes critiques : le centrage forcé (`PASSAGE_ETROIT`), l'arrêt d'urgence (`STOP`), l'amorce de virage (`VIRAGE_INTERSECTION`) et le suivi standard (`SUIVI_MUR`).

5.2 Travail demandé

1. Récupérez le script `robot_suiveur_pd.py` sur le dépôt GitHub de l'UE.
2. **Analyse de la boucle de calcul temporelle** : Au début de la méthode `boucle()`, comment la variable `dt` (intervalle de temps) est-elle calculée ? Pourquoi est-il impératif d'utiliser une mesure de temps dynamique plutôt qu'une valeur fixe (comme 0.05 s) pour calculer le terme dérivé d'un correcteur ?
3. **Fonctionnement théorique du correcteur PD** : Examinez les lignes calculant la dérivée et la commande `cmd_z`.
 - Quelle est l'influence concrète du gain proportionnel (`kp_ang`) sur le comportement du robot ?
 - Quel est le rôle géométrique exact du gain dérivé (`kd_ang`) lorsque le robot s'approche très rapidement d'un mur avec un angle d'incidence élevé ?
4. **Anticipation géométrique et Vitesse dynamique** : Dans l'état `SUIVI_MUR`, analysez le calcul de l'échantillon `erreur_anticipation` et de la variable `ratio_vitesse`. Expliquez comment ces lignes permettent au robot de « deviner » un virage avant même d'y être et d'adapter l'allure de sa course.
5. **Lissage des commandes (Filtre passe-bas)** : Expliquez le fonctionnement mathématique des lignes utilisant `self.alpha_lissage`. Quel serait le comportement visuel du robot si on réglait `alpha_lissage = 1.0` ? Et si on le réglait à `alpha_lissage = 0.01` ?
6. **À compléter dans le code** : La méthode principale `boucle(self)` a été vidée de sa logique algorithmique. En vous basant sur la structure de la machine à états décrite dans l'idée générale et sur les équations du correcteur PD et du filtre de lissage, complétez le script afin de rendre le robot pleinement opérationnel.

Finalement vous devriez obtenir :

```
user@gr-p247-humble-container:~/roskit_ws$ ros2 run robot_control test_5
[SUIVI_MUR      ] AV=1.80m | G=0.25m | D=0.39m | CmdX=0.03 | CmdZ=-0.14
[SUIVI_MUR      ] AV=1.80m | G=0.25m | D=0.38m | CmdX=0.05 | CmdZ=-0.16
[SUIVI_MUR      ] AV=1.79m | G=0.25m | D=0.38m | CmdX=0.07 | CmdZ=-0.18
```

FIGURE 5 – Resultat Test_5